keep coding

Introducción a

Javascript

keep coding

Ítalo Ravina

Tech Lead

nTeaser.

Requisitos

- VSCode: Editor de texto
- Live server extension
- Git

keep coding

Contenido

- Introducción a JS
- Crear nuestro primer archivo de js
- Variables y tipos de datos
- Operadores
- Sentencias de control (condicionales) y bloques
- Strings
- Funciones
- Arrays y Bucles
- Objetos, mutabilidad y destructuring
- Funciones avanzadas y programación funcional
- Promises
- Async / await

Qué es Javascript

JavaScript es un lenguaje de programación que permite dar vida e interactividad a las páginas web, como validar formularios que responden en tiempo real, animaciones, cargar datos sin actualizar la página, etc…Es uno de los tres pilares fundamentales del desarrollo web junto con HTML y CSS.

Introduction - JavaScript | MDN

JS más allá del navegador

Gracias a entornos como Node.js, JavaScript se ha convertido en un lenguaje completo que también funciona en el servidor (backend).

- Crear APIs
- Gestionar bases de datos
- Manejar archivos en el servidor
- …

JS breve historia

1995 - Nace JavaScript
Brendan Eich crea JavaScript en solo 10 días para Netscape Navigator. Originalmente se llamó "Mocha", luego "LiveScript".

1997 - Estandarización
Se crea ECMAScript (ES), el estándar oficial de JavaScript, para unificar el lenguaje.

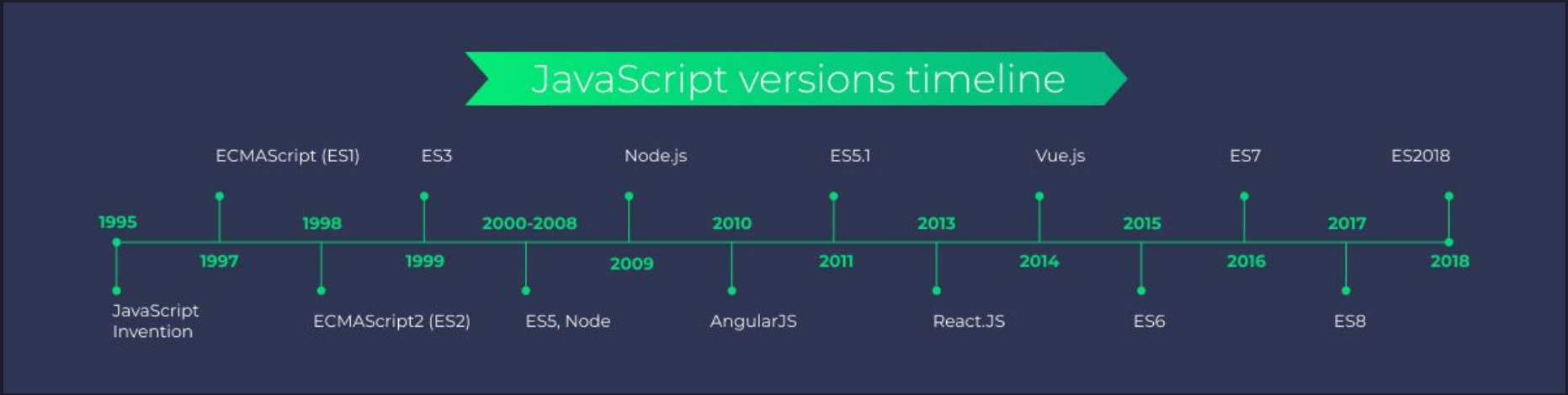
1999-2005 - La era dorada del navegador
JavaScript se populariza con AJAX, permitiendo actualizar páginas sin recargarlas. Nace la web dinámica.

2009 - JavaScript sale del navegador
Se lanza Node.js, permitiendo usar JavaScript en el servidor. El lenguaje se expande más allá del frontend.

2015 - La gran revolución (ES6/ES2015)**
Se introduce ECMAScript 2015 (ES6) con mejoras enormes: let/const, arrow functions, clases, promesas, y mucho más.
JavaScript moderno nace aquí.

2015-Actualidad - Actualizaciones anuales
Desde entonces, JavaScript se actualiza cada año con nuevas características, manteniéndose relevante y moderno.

JS timeline


JavaScript versions timeline
1995
JavaScript Invention
1997
ECMAScript (ES1)
1998
ECMAScript2 (ES2)
1999
ES3
2000-2008
ES5, Node
2009
Node.js
2010
AngularJS
2011
ES5.1
2013
React.JS
2014
Vue.js
2015
ES6
2016
ES7
2017
ES8
2018
ES2018

Creando nuestro primer JS

```
console.log("Hello World!");
```

keep coding

Sintaxis y variables

- Escritura básica de un archivo.
- Comentarios
- Palabras reservadas
- Punto y coma
- Variables (let, const y var)

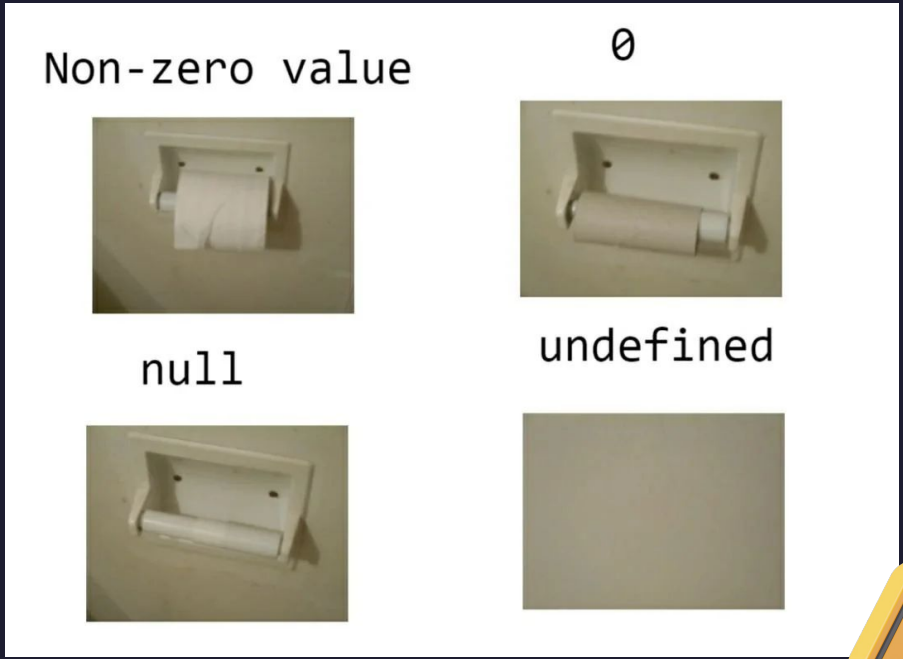
Tipos de datos

- Primitivos:
  - **Number**: 42, 3.14, -7, NaN, Infinity
  - **String**: "Keepcoding", 'Hola', `Template literal`
  - **Boolean**: true, false
  - **Undefined**: undefined (variable declarada sin valor)
  - **Null**: null (ausencia intencional de valor)
  - **Symbol**: Symbol('id') (identificador único)
  - **BigInt**: 12345678901234567890123456789 0n (números muy grandes)

- Complejos:
  - **Object**: { nombre: "Ana", edad: 25 }
  - **Array**: [1, 2, 3, 4, 5]
  - **Function**: function saludar() { }

Grammar and types - JavaScript | MDN

Null, undefined, 0…



Typeof y Operadores

- typeof: Devuelve el tipo de dato de una variable o valor
- Operadores de Asignación
- Operadores Aritméticos
- Operadores de Comparación
- Concatenación
- Operadores Lógicos
- Operador coma
- Operador de negación

Expressions and operators - JavaScript | MDN

Sentencias de control

- Condicionales:
  - if, else, else if
  - switch
- Bloques:
  - Código delimitado por llaves { }
- Hoisting
- Operador ternario

Control flow and error handling - JavaScript | MDN

Valores Truthy y Falsy

- Valores Falsy (se evalúan como false):
  - false
  - 0
  - "" (string vacío)
  - null
  - undefined
  - NaN

- Valores Truthy: Todo lo demás se evalúa como true
  - "Hola", 42, [], {}, true, etc.

Control flow and error handling - JavaScript | MDN

Strings (o cadenas de texto)

- Strings methods
  - length
  - toUpperCase()
  - toLowerCase()
  - trim()
  - charAt()
  - indexOf()
  - slice()
  - replace()
  - includes()
- Errors

Useful string methods - Learn web development | MDN

Funciones - Puntos clave

- **Declaración de funciones:** Cómo crear una función
- **Parámetros y argumentos:** Pasar datos a las funciones
- **Return**: Devolver valores
- **Scope (ámbito):** Variables locales vs globales
- **Funciones flecha:** Sintaxis moderna (ES6)
- **Funciones como valores:** Asignar funciones a variables
- **IIFE:** Funciones inmediatamente invocadas

Functions - JavaScript | MDN

Funciones

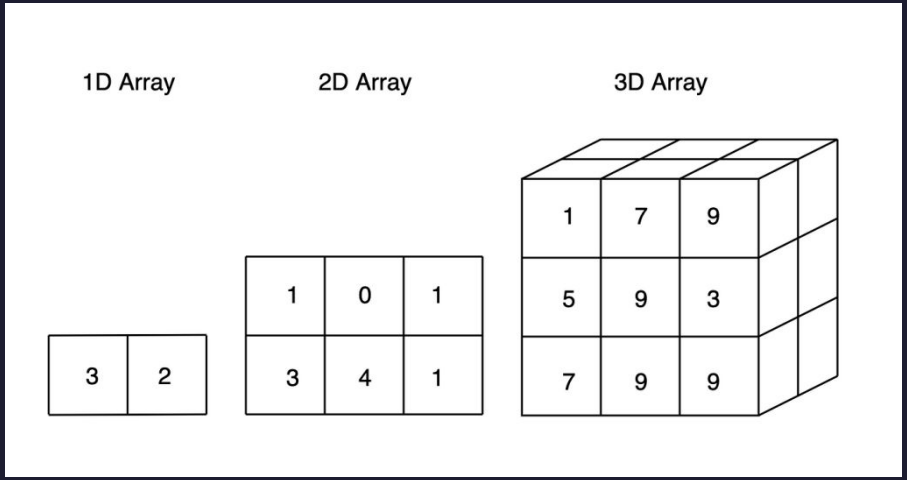
```javascript
// 1. FUNCTION DECLARATION
function greet() {
  console.log("Hello Keepcoding!");
}

// 2. PARAMETERS AND ARGUMENTS
function add(a, b) {
  console.log(a + b);
}

// 3. RETURN - Returning values
function multiply(x, y) {
  return x * y;
}
const result = multiply(4, 5);

// 4. SCOPE
let global = "I'm global";

function showScope() {
  let local = "I'm local";
  console.log(global); // Access global variable
  console.log(local);  // Access local variable
}
```

```javascript
// 5. ARROW FUNCTIONS (ES6)
const subtract = (a, b) => a - b;

// Arrow Function muliline
const myNameIs = name => {
  return `My name is ${name}!`;
};


// 6. FUNCTIONS AS VALUES
const myFunction = function (name) {
  return `Hello, ${name}`;
};
console.log(myFunction("Ana"));

// 7. IIFE (Immediately Invoked Function Expression)
(function () {
  console.log("I execute immediately");
})();

// IIFE with parameters
(function (name) {
  console.log(`Hello ${name} from an IIFE`);
})("Carlos");
```

[Functions - JavaScript | MDN]

Arrays

- Qué es?
- Crear, acceder y modificar
- Length
- Métodos básicos de arrays
- Buscar
- Arrays multi-dimensionales



Array - JavaScript | MDN

Bucles

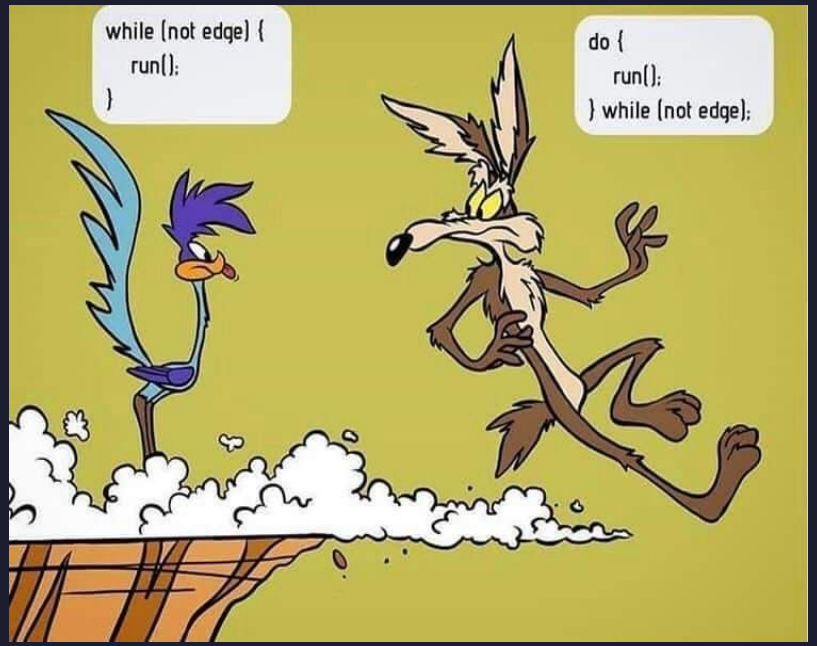
- **for clásico:** Repetir n veces con contador
- **while:** Repetir mientras se cumpla una condición
- **do...while:** Ejecutar al menos una vez
- **for...of:** Recorrer arrays (elementos)
- **break:** Salir del bucle
- **continue:** Saltar a la siguiente iteración

Loops and iteration - JavaScript | MDN

Bucles

- While:
  - Comprueba la condición
  - Si es falsa, no sigue
  - Si es verdadera, ejecuta

- Do…while:
  - Ejecuta el bloque sí o sí al menos una vez
  - Si es verdadera, repite



[Loops and iteration - JavaScript | MDN]

Objetos

- **¿Qué es un objeto?:** Colección de propiedades clave-valor
- **Crear objetos:** Notación literal
- **Acceder a propiedades:** Notación de punto y corchetes
- **Modificar y añadir propiedades:** Cambiar o crear nuevas
- **Eliminar propiedades:** Operador delete
- **Recorrer objetos:** for...in
- **Métodos útiles:** Object.keys(), Object.values(), Object.entries()

Working with objects - JavaScript | MDN

Mutabilidad, Spread y Destructuring

- **Mutabilidad:** Objetos y arrays son mutables
- **Spread operator (...):** Copiar y expandir arrays/objetos
- **Destructuring:** Extraer valores de arrays y objetos
- **Problemas comunes:** Referencias vs copias

Mutabilidad, Spread y Destructuring

```
// MUTABILIDAD:
const arr = [1, 2, 3];
arr.push(4); // Funciona (mutable)

const obj = { name: "Ana" };
obj.age = 25; // Funciona (mutable)

// SPREAD OPERATOR:
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1, 2, 3, 4, 5]

const person = { name: "Ana", age: 25 };
const newPerson = { ...person, city: "Madrid" };

// DESTRUCTURING:
// Arrays
const [a, b, c] = [1, 2, 3];

// Objects
const { name, age } = { name: "Ana", age: 25 };

// PROBLEMAS - REFERENCIAS:
const original = [1, 2, 3];
const copy = original; // ⚠️ Misma referencia
copy.push(4);
console.log(original); // [1, 2, 3, 4] ❌

// Solución con spread
const realCopy = [...original]; // ✅ Nueva copia
```

Funciones Avanzadas

- **Parámetros predeterminados:** Valores por defecto
- **Parámetros rest (...):** Agrupar argumentos en un array
- **Closures (Cierres):** Funciones que recuerdan su entorno

Closures - JavaScript | MDN

Default parameters - JavaScript | MDN

Rest parameters - JavaScript | MDN

Programación Funcional

- **¿Qué es la programación funcional?**
- **Callback functions:** Funciones pasadas como argumentos
- **forEach():** Ejecutar una función por cada elemento
- **map():** Transformar elementos y crear un nuevo array
- **filter():** Filtrar elementos que cumplan una condición
- **reduce():** Reducir el array a un solo valor

Promesas y asincronía

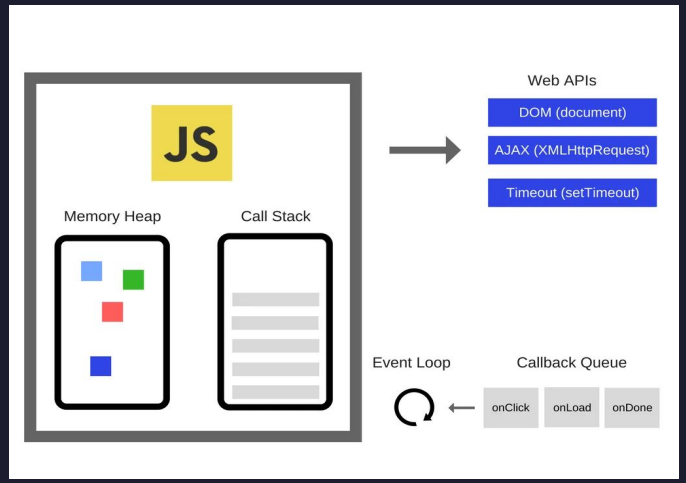
- **¿Qué es asincronía?**
- **Callbacks:** Funciones que se ejecutan al completar una tarea
- **Promises:** Representan operaciones futuras (pendiente, resuelta, rechazada)
- **async/await:** Sintaxis moderna para manejar promesas

Promise - JavaScript | MDN

async function - JavaScript | MDN

Problema de JS con el tiempo

- JS es single-threaded: ejecuta una sola cosa a la vez
- Necesitamos manejar tareas lentas como peticiones al backend o esperas
- Antes solo teníamos callbacks…y nació el temido “callback hell”




```
function hell(win) {
 // for listener purpose
 return function() {
   loadLink(win, REMOTE_SRC+'/assets/css/style.css', function() {
     loadLink(win, REMOTE_SRC+'/lib/async.js', function() {
       loadLink(win, REMOTE_SRC+'/lib/easyXDM.js', function() {
         loadLink(win, REMOTE_SRC+'/lib/json2.js', function() {
           loadLink(win, REMOTE_SRC+'/lib/underscode.min.js', function() {
             loadLink(win, REMOTE_SRC+'/lib/backbone.min.js', function() {
               loadLink(win, REMOTE_SRC+'/dev/base_dev.js', function() {
                 loadLink(win, REMOTE_SRC+'/assets/js/deps.js', function() {
                   loadLink(win, REMOTE_SRC+'/src/' + win.loader_path + '/loader.js', function() {
                     async.eachSeries(SCRIPTS, function(src, callback) {
                       loadScript(win, BASE_URL+src, callback);
                     });
                   });
                 });
               });
             });
           });
         });
       });
     });
   });
 };
}
```

Promises llegan al rescate

- Introducidas en ES (2015)
- Una promise representa: “Un valor que estará disponible ahora, más tarde o nunca”
- Estados: pending, fulfilled, rejected.

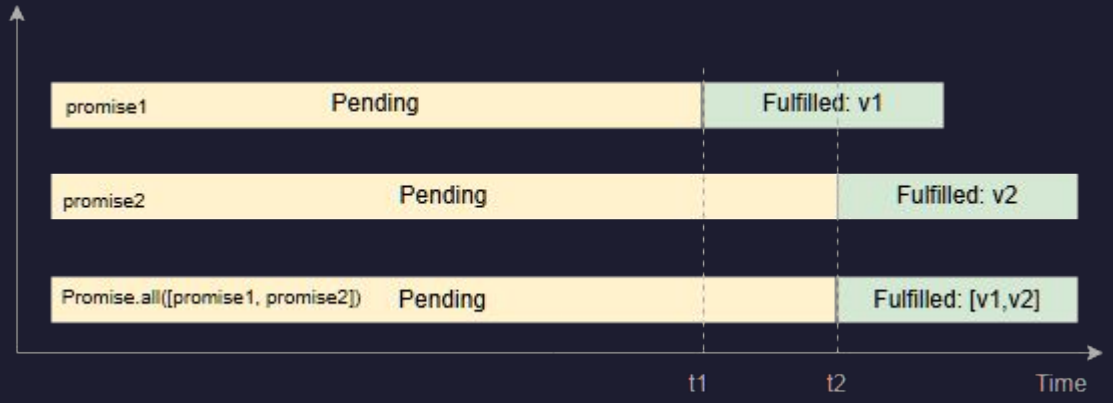
Async/await: promesas sin tanto drama

- Introducido en ES2017 (ES8).
- Permite escribir código asincrónico como si fuera síncrono.
- Facilita lectura, manejo de errores y depuración.
- No bloquea el hilo principal: await pausa la función async hasta que la promesa se resuelve, pero no bloquea el hilo principal.

```
async function planearViaje() {

  try {
    const resultado = await promesaViaje;
    console.log(resultado);
  } catch (err) {
    console.log(err)
  } finally {
    console.log("fin del plan de viaje")
  }
}

planearViaje();
```

Otras formas de manejar Promises

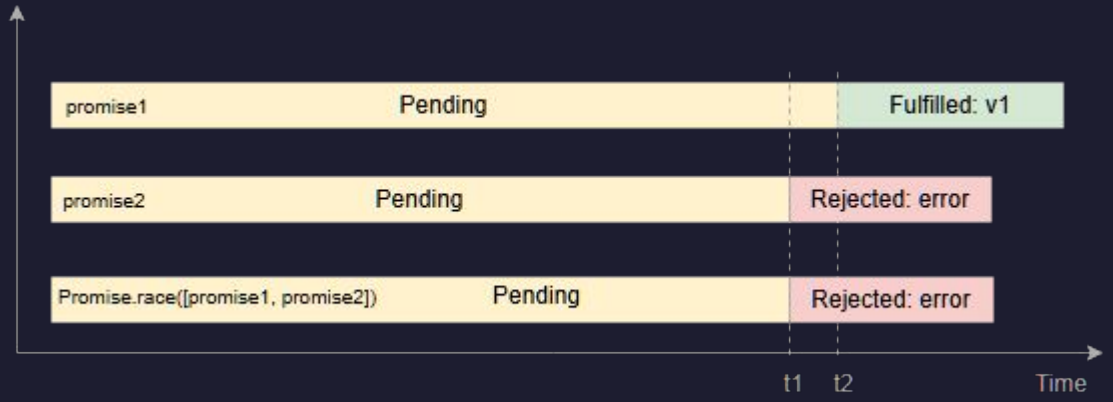
- Promise.all([p1, p2, p3]) → espera a que todas se cumplan.
  - Ejecuta varias promesas en paralelo
  - Se resuelve solo si todas se completan con éxito
  - Si una falla, falla todo



[Promise.all() - JavaScript | MDN]

Otras formas de manejar Promises

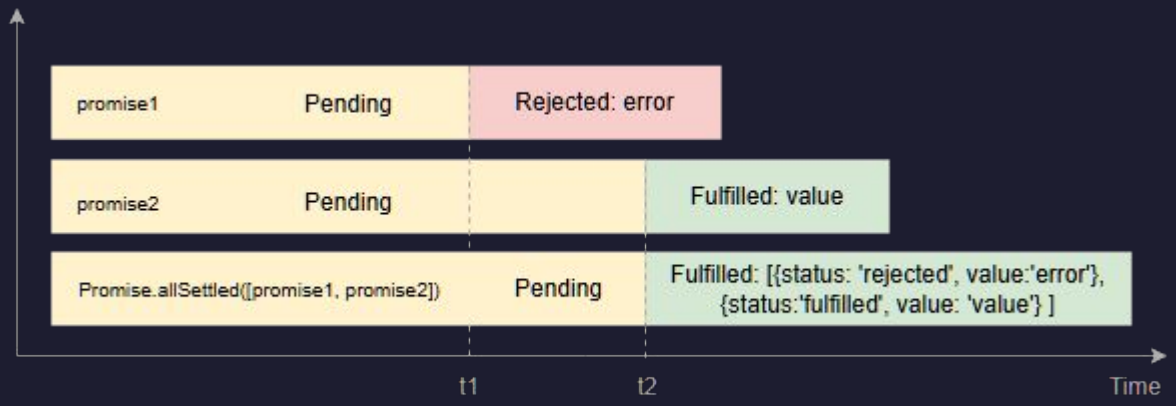
- Promise.race([p1, p2]) → la primera que responda gana.
  - Ejecuta varias promesas en paralelo
  - Se resuelve o rechaza con la primera que termine
  - No importa si las demás fallan o no



[Promise.race() - JavaScript | MDN]

Otras formas de manejar Promises

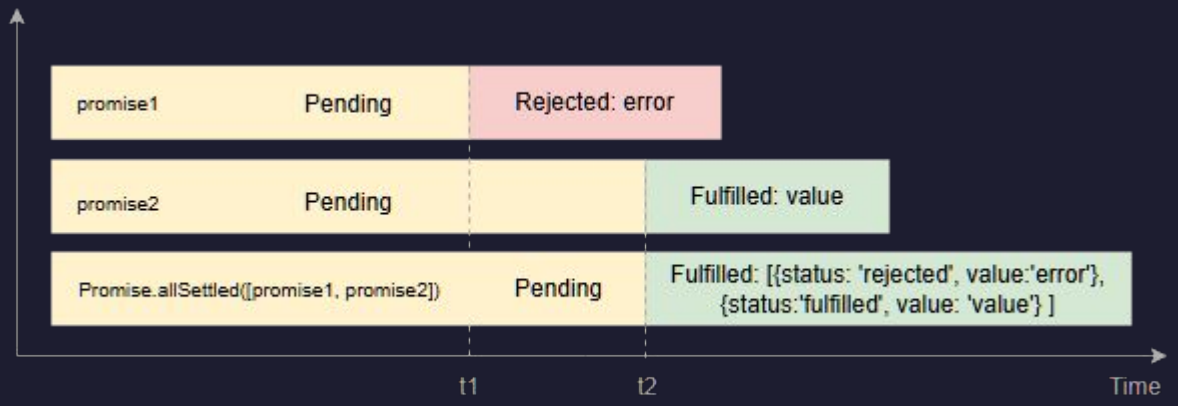
- Promise.allSettled([p1, p2]) → espera a que todas terminen (éxito o fallo).
  - Espera a que todas terminen, fallen o no
  - Nunca falla
  - Devuelve el resultado y estado de cada promesa



[Promise.allSettled() - JavaScript | MDN]

Otras formas de manejar Promises

- Promise.any([p1, p2]) → la primera que se cumpla, ignora errores.
  - Espera a la primera promesa que se resuelva correctamente
  - Ignora las que fallan
  - Falla solo si todas fallan



[Promise.any() - JavaScript | MDN]

Contenido extra

- **Dates -** Trabajar con fechas y horas -> ok
- **Regex (Expresiones Regulares) -** Validar y buscar patrones en strings -> ok
- **Modules -** Importar y exportar código (ES6 modules) -> no
- **JSON -** parse() y stringify() -> ok

keep coding

www.keepcoding.io cursos@keepcoding.io (+34) 916 33 1779